OSSP-SKILL-WEEK1

Session 1

Task1: To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Make file.

Source code:

```
GNU nano 7.2                                   Makefile
CC = gcc
CFLAGS = -Wall -Wextra

all: main

main: main.c
        $(CC) $(CFLAGS) main.c -o main

clean:
        rm -f main
```

```
#include <stdio.h>

int main() {
    printf("Hello, OSSP Skill Week 1!\n");
    printf("GCC compilation is working successfully.\n");

    return 0;
}
```

Outputs:

```
mohana@MOHANA: ~/ossp_2520090049/skill/Week1/Task1

make: Nothing to be done for 'all'.
mohana@MOHANA:~/ossp_2520090049/skill/Week1/Task1$ ./main
Hello, OSSP Skill Week 1!
GCC compilation is working successfully.
mohana@MOHANA:~/ossp_2520090049/skill/Week1/Task1$
```

```
mohana@MOHANA: ~/ossp_2520090049/skill/Week1/Task1

mohana@MOHANA:~/ossp_2520090049/skill/Week1/Task1$ nano main.c
mohana@MOHANA:~/ossp_2520090049/skill/Week1/Task1$ gcc main.c -o main
mohana@MOHANA:~/ossp_2520090049/skill/Week1/Task1$ ./main
Hello, OSSP Skill Week 1!
GCC compilation is working successfully.
mohana@MOHANA:~/ossp_2520090049/skill/Week1/Task1$
```

Verifying linux environment by uname -a:

```
mohana@MOHANA: ~

mohana@MOHANA:~$ uname -a
Linux MOHANA 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
mohana@MOHANA:~$
```

Checking GCC C compiler:

```
mohana@MOHANA: ~

mohana@MOHANA:~$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

mohana@MOHANA:~$
```

Git version:

```
mohana@MOHANA: ~

mohana@MOHANA:~$ git --version
git version 2.43.0
mohana@MOHANA:~$
```

Observation: I learned how to work with the Linux environment and verify system tools such as GCC and Git. I learned how to compile and execute a C program using GCC and understood the purpose of a Makefile for automating the compilation process. I also learned how Linux development projects can be organized and built using standard compiler and build commands.

Task2: To Analyse process abstraction, execute fork(), understand exec() family, analyse parent-child relationships, inspect process tree, practice system call tracing.

Source code:

```
GNU nano 7.2                                   skill_week1_task2.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>
```

```c
int main() {
    pid_t pid;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
        printf("Executing ls using exec()\n");

        execlp("ls", "ls", "-l", NULL);

        perror("exec failed");
        exit(1);
    } else {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent waiting for child...\n");

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

Output:

```
mohana@MOHANA: ~

mohana@MOHANA:~$ gcc skill_week1_task2.c -o skill_week1_task2
mohana@MOHANA:~$ ./skill_week1_task2
Parent Process Started
Parent PID: 638

Parent Process
Parent PID: 638
Child PID: 639
Parent waiting for child...

Child Process
Child PID: 639
Parent PID: 638
Executing ls using exec()
total 120
-rw-r--r-- 1 mohana mohana    53 Aug  8 06:04 destination.txt
drwxr-xr-x 5 mohana mohana  4096 Aug 11 06:52 ossp_2520090049
-rwxr-xr-x 1 mohana mohana    41 Aug  8 08:32 sample.txt
-rwxr-xr-x 1 mohana mohana 16312 Aug 11 07:10 skill_week1_task2
-rw-r--r-- 1 mohana mohana   879 Aug 11 07:09 skill_week1_task2.c
-rw-r--r-- 1 mohana mohana    53 Aug  8 06:03 source.txt
-rwxr-xr-x 1 mohana mohana 16528 Aug  8 04:41 week1pt1
-rw-r--r-- 1 mohana mohana  1350 Aug  8 04:41 week1pt1.c
-rwxr-xr-x 1 mohana mohana 16224 Aug  8 06:03 week2pt1
-rw-r--r-- 1 mohana mohana  1208 Aug  8 06:02 week2pt1.c
-rwxr-xr-x 1 mohana mohana 16304 Aug  8 03:26 week3pt1
-rw-r--r-- 1 mohana mohana  1023 Aug  8 03:26 week3pt1.c
-rwxr-xr-x 1 mohana mohana 16136 Aug  8 03:33 week3pt2
-rw-r--r-- 1 mohana mohana   376 Aug  8 03:32 week3pt2.c
Child process completed.
mohana@MOHANA:~$
```

Observation: I learned how Linux creates and manages processes using fork(). I understood that exec() can replace the child process with another program, while wait() allows the parent process to wait until the child process finishes.

Session2

Task1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

Source code:

```
GNU nano 7.2                                          session2_task1.c
#include <stdio.h>
#include <string.h>

int main() {
    char command[100];

    printf("OSSP Interactive Shell\n");

    while (1) {
        printf("myshell> ");

        if (fgets(command, sizeof(command), stdin) == NULL) {
            break;
        }

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        if (strlen(command) == 0) {
            continue;
        }

        printf("You entered: %s\n", command);
    }

    return 0;
}
```

Output:

```
mohana@MOHANA: ~
mohana@MOHANA:~$ cd ~
mohana@MOHANA:~$ nano session2_task1.c
mohana@MOHANA:~$ gcc session2_task1.c -o session2_task1
mohana@MOHANA:~$ ./session2_task1
OSSP Interactive Shell
myshell> hello
You entered: hello
myshell> test
You entered: test
myshell> exit
Exiting shell...
mohana@MOHANA:~$
```

Observation: I learned how to create an interactive main loop in C that repeatedly displays a prompt and reads user input. I also learned how to handle exit conditions, manage input strings, and control program flow using loops and conditional statements.

Task2: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

Source code:

```
GNU nano 7.2                                session2_task2.c
#include <stdio.h>

int main() {
    char buffer[100];
    int ch;
    int index = 0;

    printf("Type a command and press Enter:\n");
    printf("input> ");

    while ((ch = getchar()) != '\n' && ch != EOF) {
        if (ch == 127 || ch == 8) {
            if (index > 0) {
                index--;
            }
        } else if (index < 99) {
            buffer[index++] = ch;
        }
    }
```

```
    buffer[index] = '\0';

    printf("\nCommand entered: %s\n", buffer);
    printf("Characters stored: %d\n", index);

    return 0;
}
```

Output:

```
mohana@MOHANA: ~

mohana@MOHANA:~$ cd ~
nano session2_task2.c
mohana@MOHANA:~$ gcc session2_task2.c -o session2_task2
mohana@MOHANA:~$ ./session2_task2
Type a command and press Enter:
input> hello world

Command entered: hello world
Characters stored: 11
mohana@MOHANA:~$
```

Observed: I learned how to capture keyboard input in C using getchar(). I understood how to manage an input buffer to store multiple characters, process the Enter key to finish input, and handle the Backspace key by removing the last character from the buffer. I also learned how user input can be processed and displayed as a complete command.